

Arrays Notes (TUF Planly Day 3)

Topics: Intersection of Two Sorted Arrays → Majority Element-I → Leaders in an Array → Rearrange Array by Sign → Spiral Matrix

 **Color Legend** (har section mein yehi markers use hue hain)

Marker	Matlab
	Concept / Intuition
	Code
	Dry run
	Galti / Gotcha (jahan atke ya bug aaya)
	Yaad rakhne ke points
	LC practice

Index + Pattern Summary

#	Problem	Pattern	Optimal Time	Optimal Space
1	Intersection of Two Sorted Arrays	Two Pointers (merge-style)	$O(n+m)$	$O(1)$ extra
2	Majority Element-I	Moore's Voting	$O(n)$	$O(1)$
3	Leaders in an Array	Suffix scan + running max	$O(n)$	$O(n)$ answer
4	Rearrange Array by Sign	Index placement (write pointers)	$O(n)$	$O(n)$
5	Spiral Matrix	4 boundaries shrink karna	$O(n \times m)$	$O(1)$ extra

1 Intersection of Two Sorted Arrays

Problem

Do **sorted** arrays `nums1` aur `nums2` diye hain. Common elements ki list nikalni hai. Duplicates ka rule: agar `3` `nums1` mein 2 baar aur `nums2` mein 2 baar hai, to answer mein bhi 2 baar aayega (yaani **min count**).

Example: `nums1 = [1,2,3,3,4,5,6,7]`, `nums2 = [3,3,4,4,5,8]` → `[3,3,4,5]`

Intuition

Do alag events ki guest lists hain, aur dekhna hai kaun **dono** events mein hai. Lists sorted hain, to har guest ko poori doosri list mein dhoondhne ki zarurat nahi.

Brute Approach

`nums1` ke har element ke liye `nums2` mein dhoondho. Ek `visited[]` array rakho taaki `nums2` ka koi element dobara use na ho (isse duplicates sahi count hote hain).

Code (Brute)

cpp

```
vector<int> intersectionArray(vector<int>& nums1, vector<int>& nums2) {
    vector<int> ans;
    vector<int> visited(nums2.size(), 0); // nums2 ke used elements track karne ke liye
    for (int i = 0; i < nums1.size(); i++) {
        for (int j = 0; j < nums2.size(); j++) {
            if (nums1[i] == nums2[j] && visited[j] == 0) {
                ans.push_back(nums2[j]);
                visited[j] = 1; // mark used
                break; // is nums1[i] ka match mil gaya
            }
            else if (nums2[j] > nums1[i]) break; // sorted hai, aage match nahi milega
        }
    }
    return ans;
}
```

Time: $O(n \times m)$ | **Space:** $O(m)$ (`visited`)

Optimal Approach: Two Pointers

`i` `nums1` pe, `j` `nums2` pe, dono 0 se shuru. Jo **chhota hai uska pointer aage badhao**, kyunki chhote element ka match doosri list mein aage mil hi nahi sakta (sorted hai). Barabar mile to answer mein daalo aur **dono** pointers badhao.

Code (Optimal)

cpp

```
vector<int> intersectionArray(vector<int>& nums1, vector<int>& nums2) {
    vector<int> ans;
    int i = 0, j = 0;
    while (i < nums1.size() && j < nums2.size()) {
        if (nums1[i] < nums2[j]) i++; // nums1 wala chhota, aage badho
        else if (nums2[j] < nums1[i]) j++; // nums2 wala chhota, aage badho
        else { // barabar
            ans.push_back(nums1[i]);
            i++; j++;
        }
    }
    return ans;
}
```

Time: $O(n + m)$ | **Space:** $O(1)$ extra (answer count nahi kar rahe)

■ Dry Run (Optimal)

nums1 = [1,2,3,3,4,5,6,7], nums2 = [3,3,4,4,5,8]

i	j	nums1[i]	nums2[j]	Kya hua
0	0	1	3	$1 < 3, i++$
1	0	2	3	$2 < 3, i++$
2	0	3	3	barabar, push 3, $i++ j++$
3	1	3	3	barabar, push 3, $i++ j++$
4	2	4	4	barabar, push 4, $i++ j++$
5	3	5	4	$4 < 5, j++$
5	4	5	5	barabar, push 5, $i++ j++$
6	5	6	8	$6 < 8, i++$
7	5	7	8	$7 < 8, i++$
8	-	-	-	i end pe, loop band

Answer: [3,3,4,5]

■ Gotchas

- TUF ke brute wale `main` mein function ka naam `intersectionArrays` likha hai, jabki class mein `intersectionArray` hai. Naam mismatch se **compile error** aayega.
- Loop mein `&&` hai (`||` nahi), to ek array khatam hote hi loop ruk jata hai. Bacha hua part dekhne ki zarurat nahi.

■ Yaad Rakhne Ke Points

- **Pattern ka naam:** Two Pointers on sorted arrays (merge-style).
- **Sorted hona zaroori hai.** Unsorted ho to pehle sort karo ya hash/map use karo.
- Rule ek line mein: *chhota wala pointer aage, barabar to dono aage.*

■ LC Practice

350 Intersection II, 2540 Minimum Common Value, 88 Merge Sorted Array, 986 Interval List Intersections (LC 349 mein sirf unique chahiye hote hain, wo alag hai).

2 Majority Element-I

■ Problem

Array mein wo element dhoondho jo $n/2$ se zyada baar aata ho. Na mile to `-1`. Example:
`[2,2,1,1,1,2,2]` $\rightarrow$ `2`

■ Intuition

Party mein har guest ek dish laya hai, aur kisi ek dish ko aadhe se zyada guests laye hain. Dekhna hai wo kaun si dish hai.

■ Brute

Har element ke liye poora array scan karke uska count nikalo. **Time** $O(n^2)$, **Space** $O(1)$.

■ Better: HashMap

`unordered_map<int,int>` mein har element ka count rakho. Fir map scan karke `count > n/2` wala return karo. Brute mein ek hi element ka count baar-baar nikalte the, map ne wo repeat khatam kar diya.

■ Code (Better)

```
cpp
```

```
int majorityElement(vector<int>& nums) {
    int n = nums.size();
    unordered_map<int, int> mp;
    for (int num : nums) mp[num]++;           // count
    for (auto& p : mp)
        if (p.second > n / 2) return p.first;
    return -1;
}
```

Time: $O(n)$ | **Space:** $O(n)$

■ Optimal: Moore's Voting Algorithm

Ek `el` (candidate) aur `cnt` rakho:

- `cnt == 0` ho to current element ko candidate bana lo, `cnt = 1`
- element candidate ke barabar ho to `cnt++`
- alag ho to `cnt--`

Kyun chalta hai: majority element har "cancel" ke baad bhi bacha rehta hai, kyunki baaki sab elements milke bhi usse zyada nahi ho sakte.

■ Code (Optimal)

cpp

```
int majorityElement(vector<int>& nums) {
    int n = nums.size();
    int cnt = 0, el;
    for (int i = 0; i < n; i++) {
        if (cnt == 0) { cnt = 1; el = nums[i]; } // naya candidate
        else if (el == nums[i]) cnt++;
        else cnt--;
    }
    // verify pass
    int cnt1 = 0;
    for (int i = 0; i < n; i++) if (nums[i] == el) cnt1++;
    return (cnt1 > n / 2) ? el : -1;
}
```

Time: $O(n) + O(n)$ | **Space:** $O(1)$

■ Dry Run: [2, 2, 1, 1, 1, 2, 2]

i	nums[i]	Kya hua	el	cnt
0	2	cnt==0, naya candidate	2	1
1	2	barabar	2	2
2	1	alag	2	1
3	1	alag	2	0
4	1	cnt==0, naya candidate	1	1
5	2	alag	1	0
6	2	cnt==0, naya candidate	2	1

Candidate = 2. Verify mein 2 chaar baar aaya, $4 > 3$, to answer 2.

■ Gotchas (jo galtiyan hui thi)

- `if / else if / else` ek hi chain mein hona chahiye. `cnt==0` ke baad alag `if (el == nums[i])` laga diya to `cnt` 1 ki jagah 2 ho jata hai. LC 229 wale code mein ye bug aaya tha.
- `std::remove` se "delete" karne ki koshish mat karo. `remove` size nahi badalta aur peeche kachra chhod deta hai. Sach mein delete karna ho to `v.erase(remove(...), v.end())`.
- Moore's sirf **candidate** deta hai. Majority ka hona guaranteed na ho to **verify pass zaroori** hai.

■ Yaad Rakhne Ke Points

- **Pattern ka naam:** Moore's Voting (cancel karte jao, majority bachta hai).
- Ek candidate sirf $n/2$ ke liye guarantee deta hai. $n/3$ ke liye maximum **2 candidates** hote hain (kyunki 3 elements har ek $> n/3$ aate to total $> n$ ho jata), isliye do candidates aur do counts chahiye.
- LC 229 mein initial candidates ko aisi alag values se start karo jo array mein kabhi na aayein (jaise `INT_MIN`), aur match wale checks **pehle** rakho.

■ LC Practice

169 Majority Element, 229 Majority Element II

3 Leaders in an Array

Problem

Wo elements dhoondho jo apne **right wale saare elements se strictly bade** hon. **Last element hamesha leader** hota hai. Output **left-to-right** order mein chahiye. Example:

`[10, 22, 12, 3, 0, 6]` → `[22, 12, 6]`

Intuition

Parade mein ek line hai, har kisi ke paas flag pe number hai. Tum **peeche se aage** dekh rahe ho. Jis person ka number ab tak dekhe gaye sabse bade number se bada hai, wo leader hai.

Brute

Har `(i)` ke liye uske **right** wale saare `(j)` dekho. Koi `nums[j] >= nums[i]` mila to `(i)` leader nahi hai, break.

Code (Brute)

cpp

```
vector<int> leaders(vector<int>& nums) {
    vector<int> ans;
    for (int i = 0; i < nums.size(); i++) {
        bool leader = true;
        for (int j = i + 1; j < nums.size(); j++) { // j = i+1 se, sirf right side
            if (nums[j] >= nums[i]) { leader = false; break; }
        }
        if (leader) ans.push_back(nums[i]);
    }
    return ans;
}
```

Time: $O(n^2)$ | **Space:** $O(1)$ extra

Dry Run (Brute): `[1, 2, 5, 3, 1, 2]`

- `(1)` ke right mein `(2)` hai ($\geq$) → leader nahi
- `(2)` ke right mein `(5)` hai → leader nahi
- `(5)` ke right mein `(3, 1, 2)` sab chhote → **leader**
- `(3)` ke right mein `(1, 2)` chhote → **leader**
- `(1)` ke right mein `(2)` hai → leader nahi
- `(2)` last hai → **leader**

Answer: `[5, 3, 2]`

■ Optimal: Peeche se scan + running max

(mx) mein ab tak ka sabse bada rakho. Peeche se chalo. Jo element (mx) se bada ho wo leader hai, aur (mx) update karo. Order ulta aayega, to end mein (reverse) karo.

■ Code (Optimal)

cpp

```
vector<int> leaders(vector<int>& nums) {
    vector<int> ans;
    if (nums.empty()) return ans;
    int mx = nums[nums.size() - 1];
    ans.push_back(mx); // last hamesha leader
    for (int i = nums.size() - 2; i >= 0; i--) {
        if (nums[i] > mx) {
            ans.push_back(nums[i]);
            mx = nums[i];
        }
    }
    reverse(ans.begin(), ans.end()); // left-to-right order
    return ans;
}
```

Time: O(n) | **Space:** O(n) (answer ke liye)

■ Dry Run (Optimal): [10, 22, 12, 3, 0, 6]

i	nums[i]	mx	Kya hua	ans
5	6	6	last, seedha add	[6]
4	0	6	0 > 6 nahi	[6]
3	3	6	3 > 6 nahi	[6]
2	12	6	12 > 6, add, mx = 12	[6,12]
1	22	12	22 > 12, add, mx = 22	[6,12,22]
0	10	22	10 > 22 nahi	[6,12,22]

(reverse) ke baad: [22, 12, 6]

■ Gotchas (jo galtiyan hui thi)

- Brute mein (j) 0 se mat chalao, (j = i + 1) se chalao. Leader sirf apne right pe depend karta hai. Left wale elements compare ho gaye to answer galat aata hai.

- `j = -1` se loop reset karne wala trick $O(n^2)$ hai aur padhna mushkil karta hai. Clean nested loop likho.
- **Last element** alag se handle karo, warna wo miss ho jata hai.
- Condition **strictly greater** (`>`) hai. `[7, 7, 7]` pe sirf last `7` leader banta hai.
- Empty array ka check rakho, warna `nums[n-1]` crash karega.

■ Yaad Rakhne Ke Points

- **Pattern ka naam:** suffix scan (peeche se) + running max.
- Peeche se scan karoge to answer ulta nikalta hai, `reverse` zaroor lagao.
- Ye pattern bahut se problems mein dobara aata hai (running value peeche se).

■ LC Practice

1299 Replace Elements with Greatest Element on Right, 121 Best Time to Buy and Sell Stock. (LC 238 Product Except Self isi family ka hai par ek step upar, Day 5 pe Maximum Product Subarray ke saath revisit karna hai.)

4 Rearrange Array Elements by Sign

■ Problem

Array mein **barabar** number mein positive aur negative elements hain. Unhe **alternate** karo, **positive se shuru**. Dono groups ka original **relative order same** rahe. Example:

`[1, 2, -4, -5]` → `[1, -4, 2, -5]`

■ Intuition

Bachchon ko photo ke liye line mein lagana hai: red shirt, blue shirt, red, blue... Pehla position red ka hai.

■ Brute: do alag arrays

Positives aur negatives ko alag `pos` aur `neg` arrays mein daalo. Fir wapas daalo: positive **even index** `2*i`, negative **odd index** `2*i+1`.

■ Code (Brute)

```
cpp
```

```
vector<int> rearrangeArray(vector<int>& nums) {
    int n = nums.size();
    vector<int> pos, neg;
    for (int i = 0; i < n; i++) {
        if (nums[i] > 0) pos.push_back(nums[i]);
        else neg.push_back(nums[i]);
    }
    for (int i = 0; i < n / 2; i++) {
        nums[2 * i] = pos[i]; // positive even index pe
        nums[2 * i + 1] = neg[i]; // negative odd index pe
    }
    return nums;
}
```

Time: $O(n + n/2)$ | **Space:** $O(n)$

■ Optimal: ek hi pass, do write pointers

Result array `ans(n)` banao. `posIndex = 0`, `negIndex = 1`. Array **ek baar** scan karo. Positive mila to `ans[posIndex]` mein rakho aur `posIndex += 2`. Negative mila to `ans[negIndex]` mein rakho aur `negIndex += 2`.

■ Code (Optimal)

```
cpp
vector<int> rearrangeArray(vector<int>& nums) {
    int n = nums.size();
    vector<int> ans(n, 0);
    int posIndex = 0, negIndex = 1;
    for (int i = 0; i < n; i++) {
        if (nums[i] < 0) { ans[negIndex] = nums[i]; negIndex += 2; }
        else { ans[posIndex] = nums[i]; posIndex += 2; }
    }
    return ans;
}
```

Time: $O(n)$ | **Space:** $O(n)$

■ Dry Run (Optimal): [1, 2, -4, -5]

i	nums[i]	Kahan gaya	posIndex	negIndex
0	1	ans[0]	2	1
1	2	ans[2]	4	1
2	-4	ans[1]	4	3
3	-5	ans[3]	4	5

Answer: [1, -4, 2, -5]

■ Gotchas

- Ye approach tab chalti hai jab positive aur negative ki **count barabar** ho. Count alag ho to Rearrange II wala version aata hai.
- `else` mein zero bhi positive jaisa treat ho jata hai. Problem mein zero nahi hota, par dhyan rakhna.
- Optimal mein **extra array** `ans` **zaroori hai**. Original array mein hi likhoge to elements overwrite ho jayenge.

■ Yaad Rakhne Ke Points

- **Pattern ka naam:** index placement (write pointers). Isi ka idea LC 283 (Move Zeroes) aur 905 (Sort Array By Parity) mein bhi lagta hai.
- Relative order isliye same rehta hai kyunki hum **left-to-right scan** karte hain aur har group ko apne agle slot mein rakhte hain.

■ LC Practice

2149 Rearrange Array Elements by Sign (ye wahi problem hai), 283 Move Zeroes, 905 Sort Array By Parity, 75 Sort Colors

5 Print the Matrix in Spiral Manner

■ Problem

`(n × m)` matrix ko **spiral order** mein print/return karo: pehle upar ki row left→right, fir right column top→bottom, fir neeche ki row right→left, fir left column bottom→top, aur andar ki taraf repeat.

■ Intuition

Chaar boundaries socho: `top`, `bottom`, `left`, `right`. Har round mein **ek layer** (bahar ki ring) complete hoti hai, fir boundaries andar ki taraf **shrink** ho jati hain.

■ Approach (Boundary Shrinking)

1. `top = 0`, `left = 0`, `bottom = n - 1`, `right = m - 1`
2. Jab tak `top <= bottom && left <= right`:
 - **Left → Right** (`top` row), fir `top++`
 - **Top → Bottom** (`right` column), fir `right--`
 - Agar `top <= bottom`: **Right → Left** (`bottom` row), fir `bottom--`
 - Agar `left <= right`: **Bottom → Top** (`left` column), fir `left++`

■ Code

cpp

```

vector<int> spiralOrder(vector<vector<int>>& matrix) {
    vector<int> ans;
    int n = matrix.size();           // rows
    int m = matrix[0].size();        // columns
    int top = 0, left = 0;
    int bottom = n - 1, right = m - 1;

    while (top <= bottom && left <= right) {
        // 1) left -> right
        for (int i = left; i <= right; ++i)
            ans.push_back(matrix[top][i]);
        top++;

        // 2) top -> bottom
        for (int i = top; i <= bottom; ++i)
            ans.push_back(matrix[i][right]);
        right--;

        // 3) right -> left
        if (top <= bottom) {
            for (int i = right; i >= left; --i)
                ans.push_back(matrix[bottom][i]);
            bottom--;
        }

        // 4) bottom -> top
        if (left <= right) {
            for (int i = bottom; i >= top; --i)
                ans.push_back(matrix[i][left]);
            left++;
        }
    }
    return ans;
}

```

Time: $O(n \times m)$ | **Space:** $O(1)$ extra (answer count nahi kar rahe)

Dry Run

Matrix:

```

1  2  3  4
5  6  7  8
9 10 11 12
13 14 15 16

```

Round 1 ((top=0, bottom=3, left=0, right=3)):

- L→R: (1 2 3 4), (top = 1)
- T→B: (8 12 16), (right = 2)
- R→L: (15 14 13), (bottom = 2)
- B→T: (9 5), (left = 1)

Round 2 ((top=1, bottom=2, left=1, right=2)):

- L→R: (6 7), (top = 2)
- T→B: (11), (right = 1)
- R→L ((top <= bottom)): (10), (bottom = 1)
- B→T ((left <= right)): loop khali ((i = 1) se (top = 2) tak), (left = 2)

Round 3: (top (2) <= bottom (1)) false, loop band.

Answer: (1 2 3 4 8 12 16 15 14 13 9 5 6 7 11 10)

■ Gotchas

- **Teesre aur chautha loop ke (if) guards zaroori hain.** Inke bina single row ya single column wale matrix mein elements **duplicate** ho jate hain.
- (matrix[0].size()) empty matrix pe crash karega. Interview/CF mein empty ka case check kar lena.
- Har loop ke baad boundary update karna **mat bhoolo** ((top++), (right--), (bottom--), (left++)).
- Row aur column ke index ulte mat karo: (matrix[row][col]) hota hai.

■ Yaad Rakhne Ke Points

- **Pattern ka naam:** matrix boundaries (4 pointers).
- Rule: *ek side print karo, us side ki boundary andar kar do.*
- Ye same idea LC 59 (Spiral Matrix II) mein ulta use hota hai: print karne ki jagah numbers **fill** karte ho.

■ LC Practice

54 Spiral Matrix, 59 Spiral Matrix II, 48 Rotate Image

✓ Revision Checklist (agle din blank screen pe code karo)

- ☐ Intersection: two pointers (10 min)
- ☐ Majority Element-I: Moore's + verify pass (10 min)
- ☐ Leaders: peeche se scan + reverse (10 min)
- ☐ Rearrange by Sign: posIndex / negIndex (10 min)
- ☐ Spiral Matrix: 4 boundaries aur 2 `if` guards (15 min)

Jo problem blank screen pe 10-15 min mein na bane, uska ek aur LC medium karo, aur notes mein "weak" mark karo.